

Introduction au Langage C

TP n°4 : Groupe A

EPITA Cyber 1
2025-2026

1 Consignes

Le sujet du TP4 est composé de **8 exercices de programmation** en langage C.

Chaque exercice contient l'**objectif de la fonction** à écrire et des **exemples d'exécution**.

Pour chaque exercice, vous devez **soumettre un fichier ".c"** dans le **dépôt associé à l'exercice**.
Testez vos fonctions avec une fonction **main**, et **supprimez la fonction main avant de soumettre**.

→ **Votre fichier ".c" soumis ne doit plus contenir de fonction "main"**.

Les dépôts d'exercices se trouvent sur **Moodle**.

Chaque fichier .c doit commencer par un **bloc de commentaires avec vos informations personnelles** : Nom, prénom, date, et groupe (voir exemple ci-dessous).

Veillez à **déposer la version finale de votre code**.

→ **La soumission de votre code est définitive**.

Consignes additionnelles :

- Le nom du fichier .c doit être **exercice[i].c**, où [i] est à remplacer par le numéro de l'exercice, par exemple **exercice1.c** pour l'exercice numéro 1.
- Votre programme doit compiler avec la commande :
\$gcc -std=c11 -Wall -Wextra -Werror exercice1.c -o exercice1
- Respectez les prototypes des fonctions données dans l'énoncé.
- Les bibliothèques autorisées sont : stdio.h.

Exemple : Un fichier contenant un bloc d'informations personnelles.

```
1  /*  Nom: Barrot Nathanael
2     Date: 06/10
3     Groupe: A
4  */
5
6  #include <stdio.h>                                // import de stdio.h
7
8  int main(void)
9  {
10     int num=0;                                     // definition de num pour
stockage
11     printf("Entrer un entier: ");                  // affichage dans le terminal
12     scanf("%d", &num);                             // "%d" pour entrer un entier
13     return 0;
14 }
```

2 Exercice n°1 : Pointeur Affichage

Objectif :

Ecrire une fonction **affichageP** qui prend en argument un pointeur vers un entier, et **affiche dans le terminal** la valeur pointée et l'adresse de la valeur.

Le prototype de la fonction doit être :

```
1 void affichageP(int* p);
```

Exemples d'appels :

- Pour un pointeur *pA* vers un entier de valeur 13 et d'adresse 0x7fff04d1fc83, l'appel **affichage(pA)** affiche dans le terminal :

```
1 Valeur : 13
2 Adresse : 0x7fff04d1fc83
```

Conditions :

Il n'est pas nécessaire de gérer les entrées erronées de l'utilisateur.

3 Exercice n°2 : Pointeur Produit

Objectif :

Ecrire une fonction **prodP** qui prend en argument un pointeur vers un entier pA et un pointeur vers un entier pB , et **retourne** le produit des valeurs pointées par pA et pB .

Le prototype de la fonction doit être :

```
1 int prodP(int * pA, int * pB);
```

Exemples d'appels :

- Avec 2 et 5 comme valeurs pointées par pA et pB , alors **prodP(pA,pB)** retourne 10.

Conditions :

Il n'est pas nécessaire de gérer les entrées erronées de l'utilisateur.

4 Exercice n°3 : Echange via Pointeur

Objectif :

Ecrire une fonction **swapP** qui prend en argument un pointeur vers un entier pA et un pointeur vers un entier pB , et **échange les valeurs** pointées par pA et pB .

Le prototype de la fonction doit être :

```
1 void swapP(int * pA, int * pB);
```

Exemples d'appels :

- Pour 2 et 5 comme valeurs pointées par pA et pB , l'appel **swapP(pA, pB)** échange les valeurs pointées par pA et pB ; les valeurs pointées par pA et pB deviennent donc 5 et 2 respectivement.

Vérifiez à l'aide du `main()` que les valeurs des pointeurs sont bien échangées. Pourquoi sont-elles échangées alors que la fonction ne fait pas de retour ?

Conditions :

Il n'est pas nécessaire de gérer les entrées erronées de l'utilisateur.

5 Exercice n°4 : Minimum via Pointeur

Objectif :

Ecrire une fonction **minP** qui prend en argument trois pointeurs, chacun pointant vers un entier, et **retourne** le pointeur vers la valeur minimum.

Le prototype de la fonction doit être :

```
1 int * minP(int* pA, int* pB, int* pC);
```

Exemples d'appels :

- Avec 2, 9, et 5 comme valeurs pointées par pA , pB , et pC , alors **minP(pA,pB, pC)** retourne le pointeur pA .

Conditions :

Il n'est pas nécessaire de gérer les entrées erronées de l'utilisateur.

6 Exercice n°5 : Recherche Entier via Pointeur

Objectif :

Ecrire une fonction **searchValueP** qui prend en argument un pointeur *pI* vers un tableau d'entiers, la taille du tableau *n*, et une valeur *value* et **retourne** 1 si la valeur se trouve dans le tableau, et 0 sinon.

Le prototype de la fonction doit être :

```
1 int searchValueP(int* pI, int n, int value);
```

Exemples d'appels :

- Avec $tab = \{1, 2, 3\}$, `searchValueP(tab, 3, 9)` retourne 0.
- Avec $tab = \{10, 0, 5, -5\}$, `searchValueP(tab, 4, 0)` retourne 1.

Conditions :

Il n'est pas nécessaire de gérer les entrées erronées de l'utilisateur.

7 Exercice n°6 : Recherche Caractère via Pointeur

Objectif :

Ecrire une fonction **searchCarP** qui prend en argument un pointeur pS vers une chaîne de caractère, et un caractère car , et **retourne** 1 si le caractère se trouve dans la chaîne, et 0 sinon.

Le prototype de la fonction doit être :

```
1 int searchCarP(char* pS, char car);
```

Exemples d'appels :

- Avec $chaîne = "TRY"$, `searchCarP(chaîne, 'E')` retourne 0.
- Avec $chaîne = "Hi"$, `searchCarP(chaîne, 'i')` retourne 1.

Rappel :

Une chaîne de caractère se termine par le caractère nul `'\0'`.

Conditions :

Il n'est pas nécessaire de gérer les entrées erronées de l'utilisateur.

8 Exercice n°7 : Suppression Partielle

Objectif :

Ecrire une fonction **deleteHalf** qui prend en argument un pointeur vers une chaîne de caractère, et **retourne** le pointeur après avoir remplacé un caractère sur deux par le caractère '_'.

Le prototype de la fonction doit être :

```
1 char * deleteHalf(char* mot);
```

Exemples d'appels :

- Avec `char * chaine = "Mot"`, `deleteHalf(chaine)` retourne un pointeur vers la chaîne "M_t".
- Avec `char * chaine = "Minimum"`, `deleteHalf(chaine)` retourne un pointeur vers la chaîne "M_n_m_m".
- Avec `char * chaine = "abcdefgh"`, `deleteHalf(chaine)` retourne un pointeur vers la chaîne "a_c_e_g_".

Conditions :

Il n'est pas nécessaire de gérer les entrées erronées de l'utilisateur.

9 Exercice n°8 : Palindrome Pointeur

Objectif :

Ecrire une fonction **palinP** qui prend en argument un pointeur vers une chaîne de caractère, et **retourne** 1 si la chaîne de caractère forme un palindrome et 0 sinon.

Le prototype de la fonction doit être :

```
1 int palinP(char* mot);
```

Exemples d'appels :

- Avec *chaine* = " *mot* ", palinP(chaine) retourne 0.
- Avec *chaine* = " *minim* ", palinP(chaine) retourne 1.
- Avec *chaine* = " *abcCBA* ", palinP(chaine) retourne 1.

Remarque :

La fonction ne prend pas en compte la casse de lettres, autrement dit "ABba" est un palindrome..

Conditions :

Il n'est pas nécessaire de gérer les entrées erronées de l'utilisateur.